

1. I-Combinator in Javascript The I-Combinator is a function that returns its argument without changing anything (just returns the input back). It can be used when you need a function that does nothing, such as a default callback, or an unchanged transformation. However, it can also be used as a building block in combinatory logic and functional programming.

In lambda calculus, its equivalence is $\lambda x.x$

The I-Combinator, or identity function returns its argument unchanged. It is useful when used as a default transformation, callback, or building block in combinatory logic and functional programming.

Its equivalence in lambda calculus is $\lambda x.x$

2. Alpha equivalence 2.1 Which lambda expression is alpha equivalent to $\lambda x.x$: For an input x , return x $\lambda a.a$
This is basically the same, for an input a , return a

2.2 Which lambda expression is alpha equivalent to $\lambda xy.yx$: This can be reconstructed to $\lambda x. (\lambda y. (yx))$ Which means take two inputs, x and y , and then return yx $\lambda a. (\lambda b.ba)$ This equivalence just changes the letters $x \rightarrow a$ and $y \rightarrow b$, and accomplishes the same thing

2.3 Which lambda expression is alpha equivalent to $\lambda xy.xz$: This can be reconstructed to $\lambda x. (\lambda y. (xz))$ We can see that x is bound and y is bound, while z is free since no lambda introduces z $\lambda mn.mz$ This lambda equation is the same as $x \rightarrow m$ and $y \rightarrow n$, but the free z must stay as z .

3. Applying beta reduction to lambda functions 3.1 $(\lambda x.x)y$ Apply $\lambda x.x$ to the input y , Therefore $(\lambda x.x)y = y$

3.2 $\lambda x.xx$ We cannot reduce this as $\lambda x.xx$ has no input after it, it is only a function definition So, $\lambda x.xx$ is already in normal form.

If we were to give it an input and apply it to itself, we have $(\lambda x.xx)(\lambda x.xx)$

If we replace both occurrence of x in xx with the input $(\lambda x.xx)$, we get $(\lambda x.xx)(\lambda x.xx)$, which is the exact same expression. If we reduce it again, the same output will be produced, so it never reaches an answer, meaning it diverges.

3.3 $(\lambda z.zz)(\lambda y.yy)$ Replace both occurrences of z in zz with the input, $(\lambda y.yy)$ we get $(\lambda y.yy)(\lambda y.yy)$. Now, we have the same kind of function applied to itself, if we replace both occurrences of y in yy with the input $(\lambda y.yy)$, we get $(\lambda y.yy)(\lambda y.yy)$

Once again, the expression produces itself, meaning this repeats forever, so it never reaches a normal form and it diverges.

3.4 $(\lambda x.xx)y$ Replace both occurrences of x in xx with the input, y We get yy . There is no more reduction since y is just a variable, not a lambda function So, the normal form is yy .

4. Beta reduce the following expressions 4.1 $(\lambda y.zy)a$ replace y with the input a , we get za No more lambda function, so za is the normal form

4.2 $(\lambda x.x)(\lambda x.x)$ Replace the x in the first function with the input, $(\lambda x.x)$ we get $\lambda x.x$

No input is given, so $\lambda x.x$ is the normal form

4.3 $(\lambda x.xy)(\lambda x.xx)$ Replace the first x with the input, $(\lambda x.xx)$ we get $(\lambda x.xx)y$

We can apply the function again, replacing both x in xx with y we get yy The normal form is yy since we no longer have any lambda functions

4.4 $(\lambda z.z)(\lambda a.aa)(\lambda z.zb)$ Replace z with the first input, $(\lambda a.aa)$

We get $(\lambda a.aa)(\lambda z.zb)$ Replace both a in aa with the second input, $(\lambda z.zb)$ $(\lambda z.zb)(\lambda z.zb)$

Replace z in zb with the third input, $(\lambda z.zb)$ $(\lambda z.zb)b$

Replace z in zb with b , bb

no more lambda functions are present, bb is the normal form

5. Eta conversion Given a lambda function $\lambda x.(f x)$, we can simply take the function f by doing an eta reduction.

5.1 $\lambda x.zx$ $\lambda x.zx$ follows the pattern of $\lambda x.(f x)$, so we can reduce $\lambda x.zx \rightarrow z$

5.2 $\lambda x.xz$ We cannot eta reduce this function as x is being used as the function

5.3 $(\lambda x.bx)(\lambda y.ay)$ Eta reduce $(\lambda x.bx) \rightarrow b$ Eta reduce $(\lambda y.ay) \rightarrow a$

so we get ba after it is reduced

6. Which of the following are combinators 6.1 $\lambda x.xxx$ This function is a combinator as all x 's are bounded by λx

6.2 $\lambda xy.zx$ This function is not a combinator as x and y are bounded but z isn't

6.3 $\lambda xyz.xy(zx)$ This function is a combinator because all variables x , y , and z are all bounded

6.4 $\lambda xyz.xy(zxy)$ This function is a combinator because all variables x , y , and z are all bounded

7. Apply g to y -combinator $Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$

Apply g to it, so $\lambda f.(\lambda x.f(xx))(\lambda x.f(xx))g$

Replace f with g , $(\lambda x.g(xx))(\lambda x.g(xx))$

replace x with $(\lambda x.g(xx))$ $g((\lambda x.g(xx))(\lambda x.g(xx)))$

$(\lambda x.g(xx))(\lambda x.g(xx))$ is Y applied with g , so we can substitute this to $g(Y g)$

If we continue reducing this, we will get $Y g \rightarrow g(Y g) \rightarrow g(g(Y g)) \rightarrow \dots$

So, we can't simplify it further without actually knowing what g is.